Software Engineering Lab

Lab 33

Read Replica dan Replication Lag

Menjaga Read-Your-Own-Writes Tanpa Mengorbankan Skalabilitas Baca



Lukman / lukman-ss

Masalah Nyata: Ilusi Freshness di Balik Read Replica

User menulis profil baru, tapi dashboard menampilkan data lama atau kosong.

WRITE COMMITTED

Poin 1

User mengubah field 'tier' premium di endpoint PUT /profile.

WRITE COMMITTED

Poin 2

Request dialihkan ke primary, LSN increment menjadi 47.

WRITE COMMITTED

Poin 3

Write berhasil dengan status 200, response body berisi payload baru.

REPLICA LAGGING

Poin 1

WAL entry dengan LSN 47 mengalir ke replica-1 namun belum di-apply.

REPLICA LAGGING

Poin 2

Replica-1 masih menunjukkan appliedLSN = 44, data lama belum tersingkir.

REPLICA LAGGING

Poin 3

Worker goroutine replica menunggu durasi lagDuration sebelum flush.

STALE READ TERJADI

Poin 1

Load balancer mengalihkan read dashboard ke replica-1 secara round-robin.

STALE READ TERJADI

Poin 2

Query SELECT profile WHERE id=user mendapat ErrNotFound atau data usang.

STALE READ TERJADI

Poin 3

User merasa sistem rusak; actual cause adalah replication lag yang tak tertutup.

Skenario ini terjadi secara default di hampir semua database production yang menggunakan primary-replica topology. PostgreSQL streaming replication, MySQL async replication, AWS RDS read replicas, dan Azure Database for PostgreSQL semuanya menjalankan replikasi asinkron secara default tanpa konfigurasi khusus. Ketika client application melakukan pattern write-then-read pada session yang sama, ia tidak memiliki awareness bahwa data yang baru ditulis belum tersedia di replica target. Akibatnya, user mengalami anomali UX yang sulit dilacak karena log menunjukkan write sukses, tapi query berikutnya gagal membaca data yang sama.

Stale reads are not bugs; they are the expected behavior of eventually consistent replica systems.

Mengapa Read Naif ke Replica Selalu Berisiko?

Round-robin tanpa awareness LSN membuka celah read basi di setiap write window.

```
func (r *Router) ReadNaive(key string) (string, string, uint64, error) {
    replicas := r.cluster.Replicas()
    if len(replicas) == 0 {
        return r.cluster.Primary().Read(key)  // Fallback aman
    }
    idx := r.rrIndex.Add(1) % uint64(len(replicas))
    target := replicas[idx]
    return target.Read(key)  // Tanpa cek appliedLSN!
}
```

Implementasi ReadNaive menunjukkan pendekatan paling sederhana dalam read splitting: arahkan setiap request ke replica berikutnya secara berputar. Namun pendekatan ini mengabaikan fakta fundamental bahwa setiap replica memiliki appliedLSN yang mungkin jauh di belakang currentLSN primary. Dalam kondisi write-heavy workload dengan replication lag 200 milidetik, probabilitas mendapatkan stale read bisa mendekati 100% untuk key yang baru ditulis. Bug ini termasuk subtle karena tidak menghasilkan panic atau crash — hanya mengembalikan data salah yang terasa aneh bagi pengguna akhir.

KEGAGALAN FUNDAMENTAL

Tidak ada validasi AppliedLSN vs currentLSN sebelum melayani read.
Round-robin indifferent terhadap freshness state setiap replica.
Stale read probability berbanding lurus dengan throughput write dan lag duration.
Write-after-read pada session yang sama paling rentan terhadap anomali ini.

Naive load-balancing optimizes for throughput; it abandons consistency entirely.

internal/router/router.go:67–78

Mental Model: LSN sebagai Jangkar Konsistensi

Log Sequence Number membangun urutan kausal yang memungkinkan routing deterministic.

```
type WALEntry struct {
    LSN       uint64    // monotonik naik dari primary
    Key       string
    Value     string
    Timestamp time.Time
}
```

DUO KEADAAN SETIAP NODE

Primary: mengasign LSN atomic saat commit write; sumber kebenaran tunggal.
Replica: menyimpan map data KV + appliedLSN + goroutine worker consumption WAL.
Gap: delta(PrimaryLSN, ReplicaAppliedLSN) = ukuran replikasi lag saat ini.
Kunci: setiap read strategy harus meminimalkan atau menutup gap ini untuk sesi yang relevan.

Model mental ini memisahkan dua lapisan abstraction: lapisan konsistensi (LSN sequence) dan lapisan ketersediaan (data distribution). Primary bertindak sebagai single writer yang memberi nomor urut absolut pada setiap transaksi. Setiap replica secara independen consume dan apply entry WAL dengan latensi tertentu. Router membaca appliedLSN setiap replica untuk membuat keputusan routing yang informed. Tanpa LSN, kita tidak bisa membedakan replica yang fresh versus lagging secara terukur — routing menjadi guesswork belaka.

LSN is the universal causal anchor; replication lag is simply distance from that anchor.

Pipeline Write–Read dengan Replication Lag

Dari client write sampai routing decision: empat jalur strategi mitigasi.

SEKUENSI PIPELINE DISTRIBUSI

1. Client.Write(session, key, value) → Primary commit, LSN increment atomic, WAL entry broadcast ke replica channel
2. Replica worker sleep(lagDuration) → entry apply ke local data map, appliedLSN = entry.LSN, cond.Broadcast wake WaitForLSN
3a. ReadNaive: round-robin pilih replica → jika appliedLSN < lastWriteLSN → stale read atau ErrNotFound
3b. ReadWithStickySession: cek time.Since(lastWriteTime) < T_sticky → redirect ke primary; setelah TTL expired → ReadLagAware
3c. ReadWithToken: iterasi replica untuk appliedLSN >= minLSN; blocking WaitForLSN + timeout context → fallback primary
3d. ReadLagAware: hitung diff = primaryLSN - appliedLSN; filter eligible diff <= MaxLSNDiff; fallback primary jika kosong

Pipeline ini menggambarkan perjalanan lengkap sebuah request dari penulisan hingga pembacaan dengan empat varian routing strategy. Jalur pertama menelusuri write path di mana primary mengincrement currentLSN secara atomik lalu broadcasting WAL entry ke channel setiap replica. Jalur kedua mensimulasikan replication lag melalui goroutine worker yang tidur sesuai duration sebelum apply. Empat jalur ketiga menunjukkan bagaimana router memilih target read: naive round-robin tanpa proteksi, sticky session berbasis waktu, token-based LSN wait dengan blocking, atau lag-aware dynamic filtering dengan threshold.

Writes flow forward in LSN order; reads must synchronize with that order or fall back to the source.

cmd/demo/main.go:15–64 • internal/cluster/cluster.go:148–178

Implementasi Sticky Session Router

Cek lastWriteTime per session untuk mengalihkan read ke primary selama jendela sticky.

```
func (r *Router) ReadWithStickySession(sessionID, key string) (string, string, uint64, error) {
    if stateVal, ok := r.sessions.Load(sessionID); ok {
        state := stateVal.(SessionState)
        if time.Since(state.LastWriteTime) < r.config.StickyDuration {
            val, lsn, err := r.cluster.Primary().Read(key)
            return val, r.cluster.Primary().ID(), lsn, err
        }
    }
    return r.ReadLagAware(key)
}
```

Potongan kode ini merupakan implementasi time-based sticky routing yang paling sederhana namun efektif. Method pertama-tama memeriksa apakah session ID memiliki state yang terekam di sync.Map. Jika state ditemukan dan waktu sejak lastWrite masih dalam batas StickyDuration, router secara deterministik mengarahkan read ke primary — bukan replica manapun. Ini memastikan read-your-own-writes dalam jendela sticky tanpa biaya LSN check. Setelah jendela habis, flow jatuh ke ReadLagAware yang menggunakan threshold filtering.

Sticky windows trade read throughput for guaranteed freshness within a bounded temporal envelope.

Failure Path: Stale Read Anomali di Demo CLI

Output terminal membuktikan read ke replica-2 gagal meski write baru sukses di primary.

TERMINAL FAILURE LOG

[Write Primary] Wrote key 'user:profile:123', Primary LSN: 1
[Naive Read] Node: replica-2, Applied LSN: 0, Val: '', Err: key not found

Analisis:
Primary commit LSN 1 tetapi replica-2 appliedLSN masih 0 karena lag 200ms belum berlalu.
ReadNaive memilih replica-2 via round-robin tanpa pengecekan LSN.
Key tidak ditemukan karena belum di-apply di replica; user menerima ErrNotFound senyap.

DAMPAK UX

User melihat halaman kosong atau 404 setelah upload data baru.
Trust erosion: user yakin input berhasil tetapi output tidak mencerminkan input.
Debug cost tinggi karena write log sukses dan read log juga 'berhasil' (bukan error sistem).

Failure path ini diambil langsung dari demo CLI ketika ReadNaive dipanggil tepat setelah write. Primary LSN meningkat ke 1, namun replica-2 masih berada di appliedLSN 0 karena worker goroutine belum menyelesaikan sleep 200 milidetik. Load balancer round-robin memilih replica-2 untuk read berikutnya, menyebabkan key not found karena data belum tersedia. Error ini berbeda dengan koneksi mati atau timeout — aplikasi merespons 200 dengan body kosong, membuat debugging menjadi nightmare bagi engineer yang tidak memahami replication topology.

Silent correctness failures hurt more than loud crashes; they erode trust without trace.

cmd/demo/main.go:23-32 • engineering/03-execution-result.md:53-58

Mekanisme vs Gejala: Evaluasi Strategi Read Routing

Membandingkan empat pendekatan dalam hal jaminan, overhead, dan fallback behavior.

Strategi	Jaminan	Overhead	Fallback	Cocok Untuk
ReadNaive	None	Zero	Primary jika replica kosong	Development, debugging
Sticky Session	Session-level fresh	Time check O(1), session map	Lag-aware filtering	Session-bound writes
Causal Token	Causal ordering	LSN check + blocking wait	Context timeout + primary	Critical data reads
Lag-Aware	Threshold-bound	Delta LSN calculation + filter	Primary jika eligible kosong	Bulk reads, analytics

Tabel ini mengklasifikasikan empat strategi routing berdasarkan empat dimensi evaluasi. ReadNaive menghilangkan overhead komunikasi tetapi mengorbankan konsistensi sepenuhnya. Sticky session menawarkan jaminan session-level freshness dengan overhead minimal berupa time comparison dan session map lookup. Causal token memberikan jaminan kausal terukur tetapi memperkenalkan blocking wait yang dapat meningkatkan p99 latency. Lag-aware filtering menyeimbangkan throughput dan freshness melalui threshold predicate, degraded gracefully ke primary ketika tidak ada replica yang memenuhi syarat.

Every routing strategy sits on a spectrum between consistency, latency, and operational complexity.

3 Keputusan Desain Kritis Replication Strategy

Urutan keputusan arsitektur untuk menyeimbangkan consistency dan throughput.

1. TENTUKAN CONSISTENCY REQUIREMENT

Apakah session ini memerlukan read-your-own-writes (CRUD critical)?
Jika ya → sticky session atau causal token wajib digunakan.
Jika tidak → lag-aware filtering cukup untuk bulk analytics reads.

2. KALIBRASI PARAMETER WAKTU

T_sticky harus ≥ p99 replication lag aktual di production.
Default industri 5 detik bersifat heuristik; perlu tuning berbasis monitoring.
WaitTimeout pada causal token harus ≥ worst-case catch-up duration + buffer.

3. PILIH FENCING LAYER YANG TEPAT

Gunakan mode synchronous remote_apply jika freshness mutlak dibutuhkan.
Accept async replication + application-level fencing untuk throughput maksimal.
Jangan mencampuradukkan semi-sync (durability) dengan synchronous (visibility).

Tiga keputusan desain ini membentuk kerangka kerja strategis sebelum memilih implementasi teknis. Keputusan pertama menentukan level consistency requirement yang membedakan use case mana yang memerlukan protection ketat versus mana yang bisa menerima eventual consistency. Keputusan kedua menyangkut kalibrasi parameter waktu berdasarkan observasi empiris production latency distribution — bukan asumsi teoretis. Keputusan ketiga memilih lapisan proteksi yang sesuai dengan SLA bisnis, antara synchronous replication berbiaya tinggi atau application-level fencing dengan graceful degradation.

Design decisions cascade from requirements to parameters to implementation; never skip upstream thinking.

Bukti Eksekusi: Siklus Lengkap Replication & Mitigasi

Output demo CLI mengkonfirmasi empat skenario berjalan sesuai spesifikasi.

DEMO EXECUTION TRACE

```
=== Lab 33: Read Replicas and Replication Lag Demo ===

--- 1. Async Replication Lag Anomaly ---
[Write Primary] Wrote key 'user:profile:123', Primary LSN: 1
[Naive Read] Node: replica-2, Applied LSN: 0, Val: '', Err: key not found

--- 2. Sticky Session Routing ---
[Sticky Read (within 500ms)] Node: primary, LSN: 1, Val: '{...}', Err: nil
[Sticky Read (after TTL)] Node: replica-1, LSN: 1, Val: '{...}', Err: nil

--- 3. Causal Token / LSN Wait ---
[Token Read (MinLSN=2)] Node: replica-1, LSN: 2, Val: '{...}', Err: nil

--- 4. Synchronous Replication ---
[Sync Write] LSN: 1, Write Duration: 104.055166ms
[Sync Naive Read] Node: replica-2, LSN: 1, Val: '{...}', Err: nil
```

Output terminal ini menyajikan empat bukti eksekusi yang masing-masing mengonfirmasi perilaku strategi routing. Bagian pertama membuktikan anomali stale read terjadi ketika ReadNaive dipanggil pasca-write. Bagian kedua menunjukkan sticky session berhasil mengarahkan read ke primary dalam jendela 500ms, kemudian beralih ke replica setelah TTL expire. Bagian ketiga memverifikasi causal token menunggu catch-up sekitar 200ms sebelum melayani read dari replica yang sudah sesuai LSN. Bagian keempat memperlihatkan synchronous replication membutuhkan tambahan ~104ms pada write latency untuk menjamin freshness instan.

Terminal traces convert abstract consistency guarantees into observable execution evidence.

cmd/demo/main.go:1–65 • engineering/03-execution-result.md:48–73

YANG TIDAK DIBUKTIKAN

Batasan ruang lingkup laboratorium yang wajib dipahami sebelum implementasi produksi.

YANG TIDAK DIBUKTIKAN

Cluster in-memory tanpa persistence layer; tidak membuktikan recovery pasca-crash.
Tidak ada topologi cascading replica (replica-of-replica) atau multi-region replication.
Sticky 500ms dan lag 200ms bersifat ilustratif lab; bukan rekomendasi tuning produksi.
Router state disimpan di sync.Map single-process; tidak membuktikan stateless horizontal scaling.
Tidak mencakup network partition antar primary dan replica, only simulated timing lag.
p99 lag multi-region dan cross-datacenter topology tidak diuji di dalam lab.
Tidak mengimplementasikan change data capture (CDC) atau binlog-based synchronization.

Klaim teknis laboratorium ini dibatasi oleh parameter lingkungan pengujian yang disederhanakan. Cluster yang digunakan berjalan in-memory tanpa persistence disk, sehingga tidak bisa membuktikan perilaku recovery setelah node crash. State router disimpan dalam sync.Map lokal single-process, berbeda dengan arsitektur stateless production yang memerlukan Redis atau shared session store. Simulasi replication lag menggunakan timer sleep 200 milidetik, bukan network latency riil yang bervariasi. Konstanta timing seperti sticky 500ms dipilih untuk deterministik demo, bukan sebagai guideline tuning produksi.

Lab proofs establish behavioral correctness, not production-scale resilience or observability.

Trade-off Arsitektur: Freshness vs Throughput

Menilai biaya operasional penambahan routing intelligence dibandingkan benefit konsistensi.

MANFAAT & KEUNGGULAN

Read-Your-Own-Writes Guaranteed: User selalu melihat hasil tulisannya sendiri segera.
Scalable Reads: Replica menanggung sebagian besar beban read after sticky window expired.
Graceful Degradation: Fallback ke primary terjadi secara eksplisit, bukan silent failure.
Observability: LSN metrics dapat di-monitor untuk mendeteksi anomaly replication topology.

KONSEKUENSI & BIAYA

Sticky Window Overhead: Primary menerima traffic read ekstra selama T_sticky per session.
Blocking Latency: Causal token wait menambah p99 read latency saat replica belum catch-up.
Complexity Cost: Router membutuhkan session state management dan LSN inspection logic.
Fallback Concentration: Semua degraded reads terakumulasi di primary → potential bottleneck.

Penerapan strategi replication lag mitigation menuntut kompromi arsitektural antara freshness guarantee dan read throughput. Sticky session menambahkan overhead berupa concentrated primary reads selama jendela waktu, tetapi eliminasi stale read sepenuhnya untuk session-bound operations. Causal token menawarkan presisi deterministik namun memperkenalkan blocking wait yang menggeser p99 latency read path. Lag-aware filtering menyeimbangkan keduanya dengan filtering threshold yang dapat dikonfigurasi, degraded gracefully ke primary ketika semua replica terlambat. Synchronous replication menghilangkan lag sama sekali dengan harga write latency ekstra yang proporsional dengan jumlah replica.

Consistency is purchased with either latency, complexity, or primary throughput sacrifice.

Validasi Konkurensi: 15 Goroutine × 20 Operasi Race-Free

TestConcurrentAccess_RaceFree membuktikan thread-safety Router dan Cluster di bawah beban.

```
func TestConcurrentAccess_RaceFree(t *testing.T) {
    c := cluster.NewCluster(3, 10*time.Millisecond, cluster.AsyncReplication)
    defer c.Close()
    r := router.NewRouter(c, router.DefaultConfig())

    var wg sync.WaitGroup
    for i := 0; i < 15; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            sid := fmt.Sprintf('s%d', id)
            _, _ = r.Write(sid, fmt.Sprintf('k%d', id), fmt.Sprintf('v%d', id))
            for j := 0; j < 20; j++ {
                _, _, _, _ = r.ReadWithStickySession(sid, fmt.Sprintf('k%d', id))
            }
        }(i)
    }
    wg.Wait()
}
```

Uji konkurensi ini membuktikan ketahanan implementasi saat 15 goroutine bersamaan mengeksekusi pattern write-then-read pada 20 iterasi per session. Setiap goroutine menggunakan session ID unik yang menjaga isolation state di Router sync.Map. Cluster dengan tiga replica worker memproses WAL entries secara paralel dengan gap 10 milidetik. Di bawah go race detector, tidak ada data race terdeteksi di seluruh memory accesses Router methods maupun Cluster nodes. Test ini mengonfirmasi bahwa concurrent access ke shared state tetap linearizable berkat mutex proteksi di setiap node.

HASIL VERIFIKASI

Race Detector: PASS (go test -race ./... tanpa flag data race).

Session Isolation: 15 session unik tidak saling mengganggu state sync.Map window.

Cluster Stability: 3 replica workers tetap stabil tanpa goroutine leak atau deadlock.

Thread-safe shared state requires explicit mutex boundaries; race detector catches what humans miss.

tests/replication_test.go:173–210

Before vs After: Evolusi Read Routing Strategy

Transformasi dari read naif berisiko stale hingga arsitektur four-layer mitigation.

BEFORE (NAIVE ROUND-ROBIN)

Routing: Pilih replica secara acak tanpa awareness LSN state.
Failure Mode: Stale read atau ErrNotFound terjadi setiap write-then-read window.
Observability: Tidak ada metric untuk mendeteksi replication lag anomaly.
Dampak: User experience degradasi senyap tanpa error reporting yang meaningful.

AFTER (FOUR-LAYER MITIGATION)

Routing: Sticky session untuk session-bound writes, token wait untuk causal reads.
Failure Mode: Fallback eksplisit ke primary dengan logging node ID dan LSN delta.
Observability: appliedLSN per replica dapat dipromonitor untuk alerting threshold breach.
Dampak: Read-your-own-writes terjamin 100% dengan graceful degradation ketika replica lagging.

Perbandingan before-after ini menunjukkan evolusi arsitektur read routing dari pendekatan naif yang rentan stale read menuju sistem four-layer mitigation dengan fallback bertingkat. Sebelum penerapan strategi, setiap write-then-read cycle berisiko tinggi menghasilkan data usang karena tidak ada mekanisme penutupan replication gap. Setelah implementasi, sistem memiliki tiga lapis proteksi: sticky session untuk session-bound operations, causal token untuk critical reads, dan lag-aware filtering untuk bulk analytics. Setiap lapis memiliki fallback eksplisit ke primary dengan logging yang dapat diobservasi.

Before: blind distribution amplifies inconsistency. After: informed routing turns lag into a measurable variable.

Test 1: Stale Read Terjadi Tanpa Proteksi

TestNaiveReplicationLag_StaleRead membuktikan anomaly read-then-write window.

```
func TestNaiveReplicationLag_StaleRead(t *testing.T) {
    c := cluster.NewCluster(2, 200*time.Millisecond, cluster.AsyncReplication)
    defer c.Close()
    r := router.NewRouter(c, router.DefaultConfig())

    _, err := r.Write("s1", "key:A", "value-A")
    if err != nil { t.Fatalf("Write failed: %v", err) }

    // Immediately read without any mitigation
    val, nodeID, lsn, err := r.ReadNaive("key:A")
    if err == nil {
        t.Fatalf("Expected stale read error, got val=%s node=%s lsn=%d", val, nodeID, lsn)
    }
    // After waiting for lag to expire, read should succeed
    time.Sleep(250 * time.Millisecond)
    val, nodeID, lsn, err = r.ReadNaive("key:A")
    if err != nil { t.Fatalf("Post-lag read failed: %v", err) }
}
```

Unit test pertama membuktikan keberadaan stale read anomaly secara eksplisit. Setelah write ke primary dengan key 'key:A', panggilan ReadNaive seketika menghasilkan error ErrNotFound karena replica belum menerapkan WAL entry. Setelah menunggu 250 milidetik yang melampaui batas lag 200 milidetik, read yang sama berhasil mengembalikan value yang benar. Test ini mengonfirmasi bahwa anomali bersifat temporal — bukan permanen — dan dapat dicegah dengan menambahkan lapisan routing intelligence di antara write commit dan read dispatch.

VERIFIKASI INVARIAN

Immediate Read: ReadNaive pasca-write langsung menghasilkan ErrNotFound (stale read).
Post-Lag Read: Setelah sleep 250ms (> lag 200ms), read berhasil dengan nilai fresh.
Node Affinity: Setelah catch-up, read dapat berhasil dari replica manapun

Stale reads are deterministic consequences of timing gaps, not random network failures.

Test 2: Sticky Session Menjamin Read-Your-Own-Writes

TestStickySessionRouting membuktikan primary redirect dalam jendela T_sticky.

```
func TestStickySessionRouting(t *testing.T) {
    c := cluster.NewCluster(2, 200*time.Millisecond, cluster.AsyncReplication)
    defer c.Close()
    cfg := router.DefaultConfig()
    cfg.StickyDuration = 500 * time.Millisecond
    r := router.NewRouter(c, cfg)

    _, err := r.Write("s1", "key:B", "value-B")
    if err != nil { t.Fatalf("Write failed: %v", err) }

    // Read within sticky window — must hit primary
    val, nodeID, _, err := r.ReadWithStickySession("s1", "key:B")
    if err != nil || val != "value-B" || !strings.HasSuffix(nodeID, "primary") {
        t.Fatalf("Sticky read failed or hit wrong node: val=%s node=%s err=%v", val, nodeID, err)
    }

    time.Sleep(600 * time.Millisecond) // Expire sticky window
    val, nodeID, _, err = r.ReadWithStickySession("s1", "key:B")
    if err != nil || nodeID == "primary" {
        t.Fatalf("Post-sticky read should route to replica, got node=%s", nodeID)
    }
}
```

Unit test kedua mengonfirmasi bahwa sticky session routing berhasil mencegah stale read dalam jendela waktu yang ditentukan. Read yang dipanggil dalam 500 milidetik pasca-write secara konsisten diarahkan ke primary node, menjamin data fresh tersedia. Setelah jendela expire dengan sleep 600 milidetik, routing otomatis bergeser ke ReadLagAware yang menyeleksi replica berdasarkan threshold eligibility. Test ini membuktikan bahwa pendekatan time-based sederhana namun efektif untuk session-bound read patterns tanpa memerlukan LSN tracking di setiap request.

ATURAN STRICT TIMING

Time-bounded primary redirection trades read throughput for guaranteed session consistency.

Post-Expiration: Setelah 600ms, routing berubah ke ReadLagAware (replica eligible).
Value Integrity: Payload 'value-B' konsisten tersedia di primary dan replica catch-up.

Test 3: Causal Token Tunggu Catch-Up Sebelum Serve Read

TestReadWithToken_LSN membuktikan blocking wait dengan LSN deterministik.

```
func TestReadWithToken_LSN(t *testing.T) {
    c := cluster.NewCluster(1, 200*time.Millisecond, cluster.AsyncReplication)
    defer c.Close()
    r := router.NewRouter(c, router.DefaultConfig())

    lsn, err := r.Write("s1", "key:C", "value-C")
    if err != nil { t.Fatalf("Write failed: %v", err) }

    start := time.Now()
    val, nodeID, readLSN, err := r.ReadWithToken(context.Background(), lsn, "key:C")
    elapsed := time.Since(start)

    if err != nil { t.Fatalf("ReadWithToken failed: %v", err) }
    if val != "value-C" { t.Fatalf("Expected value-C, got %s", val) }
    if readLSN < lsn { t.Fatalf("Expected LSN >= %d, got %d", lsn, readLSN) }
    if elapsed < 150*time.Millisecond {
        t.Fatalf("Expected token read to wait ~200ms, finished in %v on node %s", elapsed, nodeID)
    }
}
```

Unit test ketiga menguji implementasi causal token routing yang melibatkan blocking wait mechanism. Setelah write dengan LSN tertentu, ReadWithToken iterasi replica untuk menemukan yang sudah mencapai threshold LSN. Ketika tidak ada replica yang match, method memanggil WaitForLSN pada replica pertama dengan context timeout. Test ini memvalidasi bahwa read blocked selama sekitar 200 milidetik menunggu replica catch-up, kemudian melayani response dengan nilai fresh dan LSN yang sesuai. Blocking behavior ini merupakan bedanya dengan sticky session yang hanya mengandalkan waktu tanpa memeriksa LSN aktual.

INVARIAN BLOCKING WAIT

Deterministic Wait: ReadWithToken blocking hingga replica appliedLSN >= minLSN terbukti.

Elapsed Timing: Test asserting elapsed >= 150ms mengkonfirmasi blocking behavior aktif.

Fallback Safety: Jika timeout, method fallback ke primary dengan suffix "(fallback)" dr nodeID.

Causal tokens convert temporal uncertainty into deterministic ordering guarantees.

tests/replication_test.go:100–121

Studi Kasus Industri: Replication Lag di Database Production

Bagaimana ekosistem database kelas dunia menangani stale read problem secara praktis.

AWS RDS / AURORA READ REPLICAS

Poin 1

Default: Asynchronous replication dengan lag bervariasi tergantung region dan instance size.

AWS RDS / AURORA READ REPLICAS

Poin 2

Read-Your-Own-Writes: AWS DocumentDB menyediakan 'session consistency' via sticky routing otomatis.

AWS RDS / AURORA READ REPLICAS

Poin 3

Monitoring: CloudWatch metric 'ReplicaLag' exposing seconds behind primary secara real-time.

AWS RDS / AURORA READ REPLICAS

Poin 4

Failover: Automated promotion namun read splitter perlu rekonfigurasi sticky session state.

POSTGRESQL STREAMING REPLICATION

Poin 1

Async Default: wal_level = replica, synchronous_commit = off, remote_apply belum diaktifkan.

POSTGRESQL STREAMING REPLICATION

Poin 2

Synchronous Commit: Setting synchronous_commit = remote_apply menjamin write visibility di semua replica.

POSTGRESQL STREAMING REPLICATION

Poin 3

pg_stat_replication: System view exposing replay_lsn vs write_lsn untuk monitoring lag.

POSTGRESQL STREAMING REPLICATION

Poin 4

Application Level: Driver seperti PgBouncer atau connection pooler bisa implement sticky routing.

Studi kasus industri menunjukkan bahwa masalah replication lag bukan teoretis melainkan operasional sehari-hari di infrastruktur cloud modern. AWS RDS menyediakan metric ReplicaLag melalui CloudWatch yang memungkinkan tim SRE mendeteksi anomali sebelum user complaint muncul. PostgreSQL streaming replication memberikan kontrol granular melalui synchronous_commit parameter yang membedakan antara durability guarantee dan visibility guarantee. Aplikasi enterprise besar seperti Twitter dan Netflix mengimplementasikan custom session-affinity routing di layer client library untuk menutup gap readability tanpa menunggu perbaikan di database engine.

Production systems treat replication lag as an operational variable, not an inflexible constraint.

Bagaimana WaitForLSN Membedakan Blocking dan Timeout?

Konkurensi pattern menggunakan sync.Cond + context cancellation untuk polling efisien.

```
func (n *Node) WaitForLSN(ctx context.Context, targetLSN uint64) error {
    ch := make(chan struct{})
    stop := make(chan struct{})
    go func() {
        n.mu.Lock()
        defer n.mu.Unlock()
        for n.appliedLSN < targetLSN {
            select {
            case <-stop:
                return
            default:
            }
            n.cond.Wait()  // Release lock, block until Broadcast
        }
        close(ch)
    }()
    select {
    case <-ctx.Done():
        close(stop); n.cond.Broadcast(); return ctx.Err()
    case <-ch:
        return nil
    }
}
```

Implementasi WaitForLSN menunjukkan pola konkurensi Go yang elegan untuk menunggu kondisi berbasis channel dan condvar. Goroutine waiter mengakuisisi mutex node, loop until appliedLSN memenuhi target, dan memanggil cond.Wait() yang secara atomik melepaskan mutex sambil memblokir goroutine. Worker goroutine setelah apply WAL entry memanggil cond.Broadcast() untuk membangunkan semua waiter yang sedang blocked. Context cancellation memberikan jalur escape yang aman dengan menutup stop channel dan memicu broadcast ulang agar waiter tidak stuck selamanya.

Condition variables replace polling loops; context cancellation ensures graceful deadline enforcement.

cond.Wait() melepas mutex sementara goroutine block, menghindari busy-wait CPU spinning.

internal/cluster/cluster.go:79-107

Checklist Implementasi Read Replica Strategy

10 poin verifikasi wajib sebelum merutekan traffic ke environment production.

- ✓ [ ] Tentukan consistency requirement: session-bound vs casual reads untuk memilih strategi.
- ✓ [ ] Kalibrasi T_sticky ≥ p99 replication lag aktual di production environment (bukan asumsi).
- ✓ [ ] Konfigurasi MaxLSNDiff berdasarkan observasi delta LSN normal vs anomaly threshold.
- ✓ [ ] Set WaitTimeout causal token ≥ worst-case catch-up duration + 20% buffer.
- ✓ [ ] Deploy monitoring metrics: replica lag histogram, stale read rate, primary fallback frequency.
- ✓ [ ] Implement circuit breaker untuk primary fallback ketika semua replica exceed threshold.
- ✓ [ ] Distribusikan router session state ke shared store (Redis/memcached) untuk horizontal scaling.
- ✓ [ ] Validasi bahwa sticky session map tidak memory-leak pada session idle jangka panjang.

Calibrate timing empirically, monitor continuously, and distinguish path criticality explicitly.

- ✓ [ ] Uji synchronous replication mode hanya untuk write-critical paths, bukan seluruh traffic.

Anti-Pattern: Kesalahan Desain yang Harus Dihindari

Kekeliruan arsitektur yang melumpuhkan jaminan keselamatan replication topology.

ANTI-PATTERN UMUM

Mencampuradukkan semi-sync dengan synchronous: MySQL semi-sync menjamin relay log receipt, bukan apply visibility.
Sticky Duration Terlalu Pendek: T_sticky < p99 lag menyebabkan session masih mengalami stale read selepas jendela.
Tidak Ada Fallback Primary: ReadLagAware tanpa fallback utama menyebabkan total read unavailability saat semua replica lagging.
Ignore LSN pada Cached Reads: Read dari application cache tanpa invalidasi berdasarkan LSN menghasilkan dirty reads persisten.
Hardcode Waiting Durasi: Menggunakan time.Sleep statis alih-alih context timeout menyebabkan goroutine leak dan deadlock.

CARA REPRODUKSI GAGAL DI LAB

1. Ubah router.go ReadLagAware: hapus fallback primary → test TestReplicaLagThreshold_Fallback akan fail.
2. Set MaxLSNDiff = 0 → semua replica terfilter, read selalu fallback primary; throughput read drop drastis.
3. Hapus WaitForLSN context timeout → TestReadWithToken_LSN hanging indefinitely pada replica yang crash.

Mengabaikan prinsip desain replication topology dapat berakibat fatal bagi availability dan consistency system. Anti-pattern yang paling sering terjadi adalah underestimate durasi sticky window dibanding actual replication lag, menghasilkan false sense of security bagi engineer. Tidak menyertakan fallback primary pada lag-aware routing berarti read operations akan total unavailable ketika semua replica mengalami lag spike simultan. Hardcoding sleep duration alih-alih menggunakan context timeout juga menyebabkan goroutine leakage yang dapat mematikan seluruh cluster worker.

Naive fallbacks and uncalibrated timers turn transient replication lag into catastrophic outages.

Verifikasi Mandiri: Uji & Jalankan Lab

Reproduksi seluruh bukti teknis, benchmark race detector, dan demo interaktif di terminal Anda.

```
cd labs/33-read-replicas-and-replication-lag
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

RINGKASAN METRIK LAB

Test Suite: 7 test cases lulus 100% tanpa kegagalan (2.13s total runtime).
Race Detector: Bersih dari thread contention dan data races (3.98s pass).
Dependencies: Murni Go Standard Library (sync, time, context, atomic) tanpa dependensi pihak ketiga.

Lab ini dirancang agar dapat diverifikasi secara mandiri dalam hitungan detik di mesin lokal. Seluruh implementasi dibangun murni menggunakan standard library Go tanpa memerlukan server eksternal seperti PostgreSQL atau MySQL. Perintah go test -v mengeksekusi tujuh pengujian komprehensif mulai dari anomali stale read hingga konkurensi race-free. Perintah go test -race memvalidasi keamanan thread di bawah beban 15 goroutine intensif, dan go run ./cmd/demo menampilkan demonstrasi terminal interaktif keempat skenario routing.

Zero-dependency standard library code enables instant, deterministic peer verification.

Prinsip Arsitektur: Desain untuk Asinkronitas

Pelajaran abadi dalam merancang sistem terdistribusi yang tahan terhadap replication lag.

1. ASINKRONITAS ADALAH KENYATAAN DEFAULT

Replication lag bukan anomali yang harus dihilangkan, melainkan realitas yang harus dikelola.
Asumsikan setiap read ke replica berpotensi stale kecuali dibuktikan sebaliknya oleh router.

2. CLOSING MECHANISM ADALAH HAK KLIEN

Aplikasi bertanggung jawab memilih mekanisme penutupan jarak (sticky, token, threshold).
Database engine menyediakan primitives (LSN, WAL), tetapi router aplikasi menentukan kebijakan.

3. DEGRADASI EKSPLISIT MENGALAHKAN CRASH SENYAP

Fallback ke primary dengan logging terstruktur jauh lebih baik daripada menyajikan data basi.
Transparansi status node dan LSN watermark menjaga integritas sistem jangka panjang.

Tiga prinsip arsitektur ini berlaku universal di luar implementasi lab Go ini. Pertama, sadari bahwa asinkronitas adalah default industri dan upaya menghilangkannya secara total akan mengorbankan skalabilitas tulis. Kedua, tempatkan tanggung jawab penutupan replication gap di lapisan aplikasi karena hanya aplikasilah yang memahami konteks konsistensi setiap use case. Ketiga, prioritaskan degradasi eksplisit ke primary dengan observabilitas lengkap dibandingkan membiarkan user menerima data stale secara senyap.

Asynchrony is reality; smart routing bridges the gap between scalability and user trust.

Skalabilitas Baca Terjaga, Konsistensi Terjamin.

Replication scales reads; application routing safeguards consistency.

Software Engineering Lab 33 - Read Replica dan Replication Lag

internal/cluster/cluster.go — Primary-replica simulation, WAL streaming, LSN tracking, & sync/async paths
internal/router/router.go — Four-strategy read splitter with session-affinity & dynamic lag filtering
tests/replication_test.go — 7 comprehensive test suites (stale reads, sticky routing, token wait, race-free)
cmd/demo/main.go — Complete interactive lifecycle demonstration of 4 routing scenarios

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/33-read-replicas-and-replication-lag`

Lukman / lukman-ss

internal/cluster/cluster.go — Primary-replica simulation, WAL streaming, LSN tracking, & sync/async paths
internal/router/router.go — Four-strategy read splitter with session-affinity & dynamic lag filtering
tests/replication_test.go — 7 comprehensive test suites (stale reads, sticky routing, token wait, race-free)
cmd/demo/main.go — Complete interactive lifecycle demonstration of 4 routing scenarios
Commit SHA: b41fffb
Unit Tests: 7/7 PASS (100% clean, 2.13s)
Race Detector: PASS (zero data race, 3.98s)
Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/33-read-replicas-and-replication-lag